Counting in Base 10

Theodore M. Savich

March 27, 2026

1 Diagonalizing the Count

1.1 Sublation in Counting: From Tallies to Base Systems

Counting is not merely an accumulation of marks – it is a process that both *preserves* and *transforms* prior determinations. In Hegelian terms, this movement is called *sublation* (Aufhebung), the simultaneous *negation*, *preservation*, and *uplift* of what came before. In mathematical practice, sublation is most clearly seen in the way base systems reorganize quantities into new structural units.

Consider a simple act of tally counting. If one were to count to nine using tally marks, the representation would appear as:

$$|||||||||$$

Each tally stands independently as a discrete marker of a counted object that mirrors the "world of ones" reflected in von Neumann ordinals. They could just go on and on, accumulating indefinitely. While it is more normal to represent a transformation at 5 units, let us instead live in base ten. When ten is reached, the representation undergoes an important transformation:

$$\cancel{||||||||||}$$

The previous nine marks are not erased. They are not 'gone.' But they are *negated* and *uplifted* into a new structural form. Out of the many ones, there is now one ten. This is a mathematical instance of sublation. The prior elements are not discarded. They are reorganized in a higher-level composition. The transition from loose tallies to a single "ten" does not merely introduce a new symbol; it alters how the prior marks are understood. They are still 'present,' but they no longer function as isolated entities.

So, using base systems involves "two views" of a number - but under the hood is very basic version of a diagonalizing function, δ, that lets an element reference the whole system its part of. Ten loose ones is a "many", one 10 is a "one". Diagonalization is, therefore, a way of thinking about the problem of the one and the many.

2 Understanding the Recursive Nature of Counting

Counting in base 10 involves incrementing digits and managing composition across multiple place values:

- **Units (Ones):** $10^0 = 1$
- **Tens:** $10^1 = 10$

- **Hundreds:** $10^2 = 100$
- **Thousands:** $10^3 = 1,000$, etc.

The recursive process for counting follows these steps:

1. Increment the units digit.
2. If the units digit reaches 10, reset it to 0 and increment the tens digit.
3. Repeat this process recursively for higher place values as needed.

This recursive nature allows for counting indefinitely by reusing the same increment and composition logic for each digit.

3 Why Use a Pushdown Automaton (PDA)?

A Pushdown Automaton (PDA) is suitable for modeling recursive counting due to its ability to use a stack for memory. Here's why:

- **Finite State Automaton (FSA):** Lacks the memory to handle arbitrary-length counts and composition.
- **Pushdown Automaton (PDA):** Uses a stack to provide additional memory, enabling nested operations like composition in counting.
- **Turing Machine:** While capable, it is more complex than needed for this task.

A PDA's stack can represent digit states and manage composition recursively, making it an appropriate choice.

4 Designing a PDA for Three-Digit Base-10 Counting (0-999)

While the unbounded recursive nature of counting presents challenges for standard PDA models, we can successfully design a PDA to handle counting within a fixed, practical range. This section details a Deterministic Pushdown Automaton (DPDA) capable of counting from 0 to 999, demonstrating how the stack and finite state control can manage multi-digit carries. This approach avoids the theoretical issues of infinite states or alphabets required by some conceptual models, providing a formally sound automaton for a three-digit counter.

4.1 Simulating Three Digits on One Stack

We represent the three place values (Hundreds, Tens, and Units) using distinct symbols on the PDA's single stack:

- **Units Digit Symbols:** $U_0, U_1, \ldots, U_9$
- **Tens Digit Symbols:** $T_0, T_1, \ldots, T_9$
- **Hundreds Digit Symbols:** $H_0, H_1, \ldots, H_9$

A number, represented conventionally as XYZ (X=Hundreds, Y=Tens, Z=Units), will be stored on the stack with the units digit on top. The stack configuration will be $(\#, H_X, T_Y, U_Z)$, where # is the bottom marker. For example, the number 123 would be represented by the stack $(\#, H_1, T_2, U_3)$.

4.2 Components of the Three-Digit PDA

The PDA is defined by the 7-tuple $M = (Q, \Sigma, \Gamma, \delta, q_{\text{start}}, \#, F)$:

- **States (Q):** A finite set of states manages the counting and multi-level carry logic:
  1. q_{start}: The initial state for setup.
  2. q_{idle}: The main state where the PDA resides when holding a valid count (0-999). This is the accepting state.
  3. $q_{\text{inc_tens}}$: Intermediate state entered when the units digit rolls over ($U_9 \to U_0$), signaling the need to process the tens digit via an epsilon transition.
  4. $q_{\text{inc_hundreds}}$: Intermediate state entered when the tens digit rolls over ($T_9 \to T_0$), signaling the need to process the hundreds digit via an epsilon transition.
  5. q_{halt}: A non-accepting state entered when an attempt is made to increment the count beyond 999 (hundreds digit rollover).
- **Input Alphabet (Σ):** Contains a symbol representing one unit to be counted, plus the empty string ϵ for internal transitions.

$$\Sigma = \{\text{tick}, \epsilon\}$$

- **Stack Alphabet (Γ):** Includes the bottom marker and symbols for each digit in each place value.

$$\Gamma = \{\#, H_0, \ldots, H_9, T_0, \ldots, T_9, U_0, \ldots, U_9\}$$

- **Transition Function (δ):** Defined formally in Section 8.
- **Initial State (q_0):** q_{start}.
- **Initial Stack Symbol (Z_0):** $\#$. (Implicitly placed on the stack at start).
- **Final States (F):** Only the state representing a valid count within the range is accepting.

$$F = \{q_{\text{idle}}\}$$

4.3 Automaton Behavior

The three-digit counter operates as follows:

1. **Initialization:** Start in q_{start}. On an epsilon transition seeing $\#$, push H_0, T_0, U_0 onto the stack (representing 0) and transition to q_{idle}. Stack: $(\#, H_0, T_0, U_0)$.
2. **Counting (Units Increment):** In state q_{idle}, read a 'tick' input.
   - If the top symbol is U_n where $n < 9$, pop U_n, push U_{n+1}, and remain in q_{idle}.
   - If the top symbol is U_9, pop U_9, push nothing. Transition to $q_{\text{inc_tens}}$ to handle the carry to the tens place. The T_Y symbol is now exposed on top.
3. **Carry Handling (Tens Increment):** In state $q_{\text{inc_tens}}$, perform an epsilon transition based on the exposed tens digit T_m:

- If the top symbol is T_m where $m < 9$, pop T_m. Push T_{m+1}, then push U_0. Transition back to q_{idle}. The carry is complete. Stack: $(\#, H_X, T_{m+1}, U_0)$.
- If the top symbol is T_9, pop T_9, push nothing. Transition to $q_{\text{inc_hundreds}}$ to handle the carry to the hundreds place. The H_X symbol is now exposed on top.

4. **Carry Handling (Hundreds Increment):** In state $q_{\text{inc_hundreds}}$, perform an epsilon transition based on the exposed hundreds digit H_k:
   - If the top symbol is H_k where $k < 9$, pop H_k. Push H_{k+1}, then push T_0, then push U_0. Transition back to q_{idle}. The carry is complete. Stack: $(\#, H_{k+1}, T_0, U_0)$.
   - If the top symbol is H_9 (representing an attempt to increment 999), pop H_9. Push H_0, then push T_0, then push U_0. Transition to the non-accepting state q_{halt}. Stack: $(\#, H_0, T_0, U_0)$.
5. **Halt State:** Once in q_{halt}, no further transitions are defined. The machine halts and implicitly rejects any further input, indicating overflow beyond 999.

5 State Diagram for Three-Digit Counter

The diagram illustrates the states and key transitions. Multi-symbol stack operations are abbreviated (e.g., $T_m \to U_0, T_{m+1}$ means pop T_m, push T_{m+1}, push U_0).

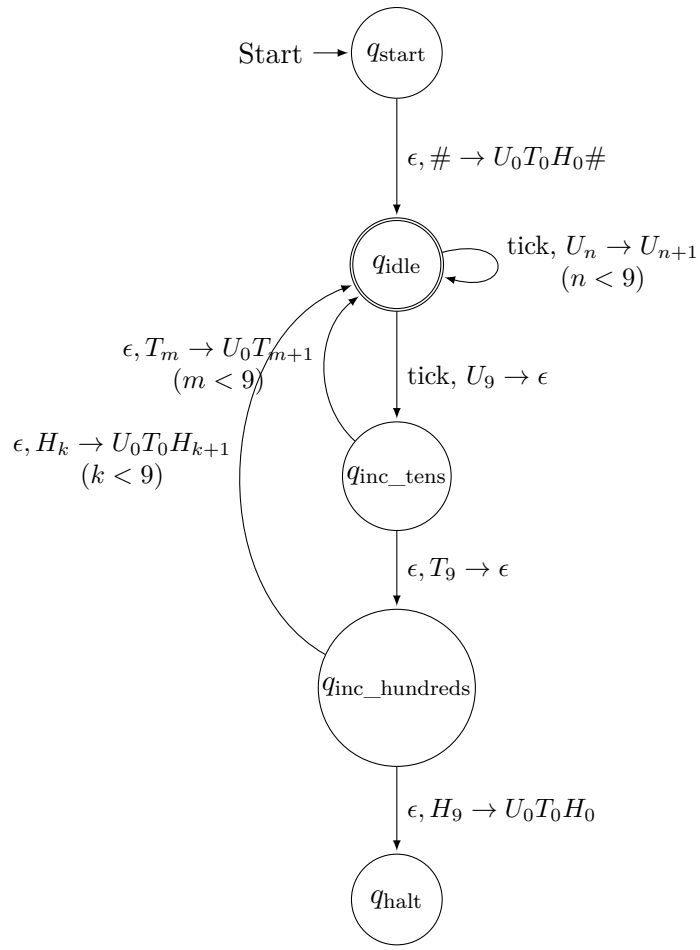

Figure 1: State Diagram for the Three-Digit (0-999) Counter PDA

6 Detailed Example Execution: Counting from 98 to 101

This section demonstrates handling carries across two place values.

1. **Start at 98:** Assume the PDA is in state q_{idle} with stack $(\#, H_0, T_9, U_8)$.
2. **Input 99 ('tick'):**
   - In q_{idle}, reads 'tick'. Top is U_8. Pops U_8, pushes U_9. Stays q_{idle}.
   - Stack: $(\#, H_0, T_9, U_9)$ (represents 99).
3. **Input 100 ('tick'):**
   - In q_{idle}, reads 'tick'. Top is U_9. Pops U_9, pushes nothing. Enters $q_{\text{inc_tens}}$.
   - Stack: $(\#, H_0, T_9)$.
   - Epsilon transition from $q_{\text{inc_tens}}$. Top is T_9. Pops T_9, pushes nothing. Enters $q_{\text{inc_hundreds}}$.
   - Stack: $(\#, H_0)$.
   - Epsilon transition from $q_{\text{inc_hundreds}}$. Top is H_0. Pops H_0. Pushes H_1, then T_0, then U_0. Enters q_{idle}.

- Stack: $(\#, H_1, T_0, U_0)$ (represents 100).

4. **Input 101 ('tick'):**
   - In q_{idle}, reads 'tick'. Top is U_0. Pops U_0, pushes U_1. Stays q_{idle}.
   - Stack: $(\#, H_1, T_0, U_1)$ (represents 101).

7 Handling Multi-Level Carries

This PDA manages carries across multiple digits using intermediate states:

1. **Units Carry:** U_9 rollover triggers a transition to $q_{\text{inc_tens}}$, popping U_9 and exposing the tens digit T_Y.
2. **Tens Processing:** $q_{\text{inc_tens}}$ handles T_Y via epsilon transition.
   - If $Y < 9$, it increments T_Y to T_{Y+1}, pushes the U_0, and returns control to q_{idle}.
   - If $Y = 9$, it pops T_9 and transitions control to $q_{\text{inc_hundreds}}$, exposing the hundreds digit H_X.
3. **Hundreds Processing:** $q_{\text{inc_hundreds}}$ handles H_X via epsilon transition.
   - If $X < 9$, it increments H_X to H_{X+1}, pushes T_0, pushes U_0, and returns control to q_{idle}.
   - If $X = 9$, it pushes H_0, T_0, U_0 (representing the rollover part) and transitions to q_{halt} to signal overflow.

This chained state transition correctly simulates the ripple effect of a carry across units, tens, and hundreds places.

8 Formal Transition Function (δ) for Three-Digit Counter

The transition function $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times \Gamma \to Q \times \Gamma^*$ is defined as: *(Note: Stack push $(S_1, S_2, \dots)$ pushes S_n first, ..., S_2, then S_1 last/top)*

- **Initialization:**
$$\delta(q_{\text{start}}, \epsilon, \#) = (q_{\text{idle}}, (U_0, T_0, H_0, \#))$$
- **Idle State (Units):** For $n \in \{0, \dots, 8\}$:
$$\delta(q_{\text{idle}}, \text{tick}, U_n) = (q_{\text{idle}}, (U_{n+1}))$$
$$\delta(q_{\text{idle}}, \text{tick}, U_9) = (q_{\text{inc_tens}}, ())$$
- **Tens Carry State (Epsilon):** For $m \in \{0, \dots, 8\}$:
$$\delta(q_{\text{inc_tens}}, \epsilon, T_m) = (q_{\text{idle}}, (U_0, T_{m+1}))$$
$$\delta(q_{\text{inc_tens}}, \epsilon, T_9) = (q_{\text{inc_hundreds}}, ())$$
- **Hundreds Carry State (Epsilon):** For $k \in \{0, \dots, 8\}$:
$$\delta(q_{\text{inc_hundreds}}, \epsilon, H_k) = (q_{\text{idle}}, (U_0, T_0, H_{k+1}))$$
$$\delta(q_{\text{inc_hundreds}}, \epsilon, H_9) = (q_{\text{halt}}, (U_0, T_0, H_0))$$
- **Halt State:**
$$\delta(q_{\text{halt}}, \cdot, \cdot) = \emptyset$$

9 Example: Counting from 998 to Overflow

Tracing the behavior at the upper limit:

1. **Start at 998:** State q_{idle}, Stack $(\#, H_9, T_9, U_8)$.
2. **Input 999 ('tick'):**
   - $\delta(q_{\text{idle}}, \text{tick}, U_8) = (q_{\text{idle}}, (U_9))$.
   - Stack: $(\#, H_9, T_9, U_9)$ (represents 999). State q_{idle}.
3. **Input 1000 ('tick'):**
   - $\delta(q_{\text{idle}}, \text{tick}, U_9) = (q_{\text{inc_tens}}, ())$.
   - Stack: $(\#, H_9, T_9)$. State $q_{\text{inc_tens}}$.
   - ϵ-trans: $\delta(q_{\text{inc_tens}}, \epsilon, T_9) = (q_{\text{inc_hundreds}}, ())$.
   - Stack: $(\#, H_9)$. State $q_{\text{inc_hundreds}}$.
   - ϵ-trans: $\delta(q_{\text{inc_hundreds}}, \epsilon, H_9) = (q_{\text{halt}}, (U_0, T_0, H_0))$.
   - Stack: $(\#, H_0, T_0, U_0)$. State q_{halt}.
4. **State q_{halt}:** The PDA enters the non-accepting halt state. The stack represents '000', but the state signals the overflow. Further 'tick' inputs are rejected as no transitions are defined from q_{halt}.

10 Practical Considerations and Limitations

This three-digit counter PDA successfully models counting within its defined range (0-999).

- **Bounded Range:** The automaton is explicitly designed for three digits.
- **Scalability Limitation (State-Based):** Extending this specific design to, say, ten digits would require ten distinct place-value symbol sets $(U, T, H, Th, \dots)$ and ten corresponding intermediate carry states $(q_{\text{inc_tens}}, q_{\text{inc_hundreds}}, \dots)$. While possible, the number of states and transitions grows linearly with the number of digits, making the explicit definition cumbersome for very large, fixed ranges.
- **Contrast with Unbounded Models:** This successful bounded model highlights why the unbounded recursive counter using only simple digit symbols $(D_0..D_9)$ and few states failed with standard PDAs. Managing the carry requires either distinct symbols/stack structure per position or distinct states per carry level, which becomes infinite in the unbounded case unless a more powerful model (like a Turing Machine) is used.
- **Output Interpretation:** Reading the final count involves interpreting the stack configuration $(\#, H_X, T_Y, U_Z)$.

11 Conclusion

By extending the logic used for the two-digit counter, we have designed a formally correct Deterministic Pushdown Automaton capable of counting from 0 to 999. This PDA uses distinct stack symbols for each place value (Units, Tens, Hundreds) and employs intermediate states ($q_{\text{inc_tens}}$, $q_{\text{inc_hundreds}}$) to manage the propagation of carries across digit boundaries via epsilon transitions. The model correctly handles multi-digit increments and explicitly halts in a non-accepting state upon overflow, demonstrating that standard PDAs can effectively model counting within a fixed, multi-digit range.

This contrasts with attempts to model unbounded counting using simpler stack representations, confirming that the specific way carries interact with place value requires careful state or stack management that becomes infinite in the unbounded case for standard PDAs. This exercise validates the suitability of PDAs for such bounded counting tasks while illustrating the design patterns needed to handle multi-level dependencies using finite state control and stack manipulation.

11.1 Key Takeaways

- **Fixed-Range Counting with PDAs:** Standard DPDAs can correctly model multi-digit base-10 counting within a predefined range (e.g., 0-999).
- **Hierarchical Carry Management:** Multi-level carries can be managed using a chain of intermediate states, each responsible for processing the carry at a specific place value via epsilon transitions.
- **Stack Representation:** Using distinct symbols for each place value (e.g., H_k, T_m, U_n) is crucial for the state logic to correctly identify and process the appropriate digit during carries.
- **Scalability vs. Boundedness:** While this state-based approach works, its complexity grows with the number of digits, making it practical for bounded ranges but unsuitable for theoretically unbounded counting, which is better modeled by Turing Machines or alternative formalisms.

Python Test Script (0-999)

12 Counting On and Counting Back

Counting on (incrementing) and counting back (decrementing) are just two sides of the same process of *composing* or *decomposing* our base10 units. When we count on by one tick, we *compose a base*: we add one unit symbol on top of the stack, and when that unit position reaches 10, we reorganizecomposethe ten units into one ten. Conversely, when we count back by one tock, we *decompose a base*: we remove one unit, and if the units position drops below zero, we borrow by decomposing one ten into ten ones, cascading as necessary.

12.1 Formal Description

We extend our DPDA tuple

$$M = \big(Q,\ \Sigma,\ \Gamma,\ \delta,\ q_{\text{start}},\ Z_0,\ F\big)$$

with

$$
\begin{aligned}
Q &= \{\, q_{\text{start}},\, q_{\text{idle}},\, q_{\text{inc_tens}},\, q_{\text{inc_hundreds}},\, q_{\text{dec_tens}},\, q_{\text{dec_hundreds}},\, q_{\text{halt}},\, q_{\text{underflow}} \}, \\
\Sigma &= \{\, \text{tick},\ \text{tock},\ \epsilon \}, \\
\Gamma &= \{\, \#,\, U_0, \ldots, U_9,\, T_0, \ldots, T_9,\, H_0, \ldots, H_9 \}, \\
Z_0 &= \#, \quad F = \{\, q_{\text{idle}} \,\}.
\end{aligned}
$$

12.2 State Table

State	**Input**	**Top of Stack**	**Pop/Push**	**Next State**
q_{start}	ϵ	$\#$	push $(U_0, T_0, H_0, \#)$	q_{idle}
Counting On (tick): compose a base at each place value				
q_{idle}	tick	U_n $(n < 9)$	pop U_n, push U_{n+1}	q_{idle}
q_{idle}	tick	U_9	pop,	$q_{\text{inc_tens}}$
$q_{\text{inc_tens}}$	ϵ	T_m $(m < 9)$	pop T_m, push (U_0, T_{m+1})	q_{idle}
$q_{\text{inc_tens}}$	ϵ	T_9	pop,	$q_{\text{inc_hundreds}}$
$q_{\text{inc_hundreds}}$	ϵ	H_k $(k < 9)$	pop H_k, push (U_0, T_0, H_{k+1})	q_{idle}
$q_{\text{inc_hundreds}}$	ϵ	H_9	pop, push (U_0, T_0, H_0)	q_{halt}
Counting Back (tock): decompose a base at each place value				
q_{idle}	tock	U_n $(n > 0)$	pop U_n, push U_{n-1}	q_{idle}
q_{idle}	tock	U_0	pop,	$q_{\text{dec_tens}}$
$q_{\text{dec_tens}}$	ϵ	T_m $(m > 0)$	pop T_m, push (U_9, T_{m-1})	q_{idle}
$q_{\text{dec_tens}}$	ϵ	T_0	pop,	$q_{\text{dec_hundreds}}$
$q_{\text{dec_hundreds}}$	ϵ	H_k $(k > 0)$	pop H_k, push (U_9, T_9, H_{k-1})	q_{idle}
$q_{\text{dec_hundreds}}$	ϵ	H_0	pop, push (U_9, T_9, H_9)	$q_{\text{underflow}}$
q_{halt}				
$q_{\text{underflow}}$				

12.3 Circular State Diagram

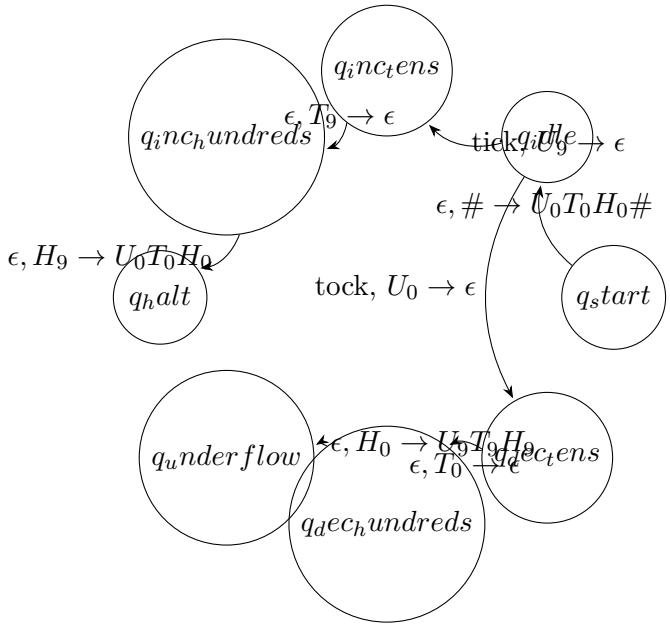

Figure 2: Circular Layout of the Extended Up/Down (0999) DPDA

Python Test Script Counting On and Back